

WEEK-4 PRACTICAL

Task 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

CODE:

```
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int pid[3], status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    for (int i = 0; i < 3; i++)
    {
        pid[i] = fork();

        if (pid[i] == 0)
        {
            printf("Child %d started. PID = %d, Parent PID = %d\n",
                i + 1, getpid(), getppid());

            sleep(i + 1);
            exit(10 + i);
        }
    }

    printf("All 3 child processes have been created.\n");

    printf("Using wait():\n");
    wait(&status);

    printf("wait() detected child PID finished with exit status %d\n",
        WEXITSTATUS(status));

    printf("Using waitpid():\n");

    for (int i = 0; i < 3; i++)
    {
        waitpid(pid[i], &status, 0);

        printf("waitpid() detected child PID %d finished with exit status %d\n",
            pid[i], WEXITSTATUS(status));
    }

    printf("Parent process completed.\n");

    return 0;
}
```

OUTPUT;

```
root@Mani: ~  
root@Mani:~# nano ProcessParent.c  
root@Mani:~# gcc ProcessParent.c -o ProcessParent  
root@Mani:~# ./ProcessParent  
Parent Process Started  
Parent PID: 551  
Child 1 started. PID = 552, Parent PID = 551  
Child 2 started. PID = 553, Parent PID = 551  
All 3 child processes have been created.  
Using wait():  
Child 3 started. PID = 554, Parent PID = 551  
wait() detected child PID finished with exit status 10  
Using waitpid():  
waitpid() detected child PID 552 finished with exit status 10  
waitpid() detected child PID 553 finished with exit status 11  
waitpid() detected child PID 554 finished with exit status 12  
Parent process completed.  
root@Mani:~#
```

REPORT: The Multiple child processes were created successfully and their completion was synchronized using wait() and waitpid().

Task 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

CODE:

```
root@Mani: ~  
GNU nano 7.2 ChildProcess.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <stdlib.h>  
  
int main()  
{  
    int pid = fork();  
  
    if (pid == 0)  
    {  
        printf("Child process: PID = %d\n", getpid());  
        printf("Child process terminating...\n");  
        exit(0);  
    }  
    else  
    {  
        printf("Parent process: PID = %d\n", getpid());  
        printf("Child PID = %d\n", pid);  
  
        sleep(10);  
  
        printf("Parent terminating...\n");  
    }  
  
    return 0;  
}
```

OUTPUT:

```
root@Mani:~# nano ChildProcess.c  
root@Mani:~# gcc ChildProcess.c -o ChildProcess  
root@Mani:~# ./ChildProcess  
Parent process PID = 638  
Child PID = 631  
Child process' PID = 631  
Child process terminating...  
Parent terminating...  
root@Mani:~#
```

REPORT: The zombie process was successfully observed, and using wait() in the parent process prevented the child from remaining as a zombie.

PS -EL:

```
root@Mani: ~
root@Mani:~# ps -el
F S      UID      PID      PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S      0         1         0  0  80   0 -  5446 do_epo ?           00:00:02 systemd
5 S      0         2         1  0  80   0 -   795 poll_s hvc0       00:00:00 init-systemd(Ub
0 S      0         6         2  0  80   0 -   799 unix_s hvc0       00:00:00 init
4 S      0        52         1  0  79  -1 - 12590 do_epo ?           00:00:02 systemd-journal
4 S      0        99         1  0  80   0 -  6252 do_epo ?           00:00:00 systemd-udev
4 S    991       151         1  0  80   0 -  5365 do_epo ?           00:00:00 systemd-resolve
4 S    996       168         1  0  80   0 - 22757 do_epo ?           00:00:00 systemd-timesyn
4 S      0       174         1  0  80   0 -  1061 hrtime ?           00:00:00 cron
4 S    101       175         1  0  80   0 -  2399 do_epo ?           00:00:00 dbus-daemon
4 S      0       185         1  0  80   0 -  4493 do_epo ?           00:00:00 systemd-logind
4 S    102       195         1  0  80   0 - 55629 poll_s ?           00:00:00 rsyslogd
4 S      0       207         1  0  80   0 -   781 poll_s tty1       00:00:00 agetty
4 S      0       208         1  0  80   0 - 26754 poll_s ?           00:00:00 unattended-upgr
5 S      0       294         2  0  80   0 -   796 vsock_ ?           00:00:00 SessionLeader
5 S      0       295       294  0  80   0 -   800 poll_s ?           00:00:00 Relay(296)
4 S      0       296       295  0  80   0 -  1520 do_wai pts/0       00:00:00 bash
4 S      0       297         2  0  80   0 -  1669 do_wai pts/1       00:00:00 login
4 S      0       340         1  0  80   0 -  5026 do_epo ?           00:00:00 systemd
5 S      0       341       340  0  80   0 -  5289 do_sig ?           00:00:00 (sd-pam)
4 S      0       363       297  0  80   0 -  1520 poll_s pts/1       00:00:00 bash
4 S      0       679       296  0  80   0 -  2335 poll_s pts/0       00:00:00 top
4 S      0      4969         1  0  80   0 -   702 do_wai ?           00:00:00 apt.systemd.dai
0 S      0      4973      4969  0  80   0 -   702 do_wai ?           00:00:00 apt.systemd.dai
4 S      0      5336         1  0  80   0 -  92515 poll_s ?           00:00:00 packagekitd
4 S    990      5341         1  0  80   0 -  77042 poll_s ?           00:00:00 polkitd
4 S      0      5450      4973 45  80   0 -  89433 poll_s ?           00:00:55 unattended-upgr
4 S     42      5849      5450  0  80   0 -   5075 poll_s ?           00:00:00 http
4 S     42      5850      5450  2  80   0 -   5075 poll_s ?           00:00:01 http
5 S      0      6133         2  0  80   0 -   796 vsock_ ?           00:00:00 SessionLeader
5 S      0      6134      6133  0  80   0 -   800 poll_s ?           00:00:00 Relay(6141)
4 S      0      6141      6134  0  80   0 -  1520 do_wai pts/2       00:00:00 bash
4 R      0      6217      6141  0  80   0 -  2080 -          pts/2       00:00:00 ps
root@Mani:~#
```

REPORT: The process table was successfully observed using ps -el, and the zombie process was identified by the Z state.